Seuillage

Pr. Issam QAFFOU

Plan

- Introduction
- Définition
- Utilité
- Types de seuillage
- Application
- Limites

Introduction

- En traitement d'images, l'un des objectifs principaux est de **séparer l'objet d'intérêt (avant-plan)** du **fond (arrière-plan)**. Le **seuillage** est l'une des méthodes les plus simples et les plus efficaces pour réaliser cette segmentation.
- Il repose sur l'idée que l'objet et le fond ont des **niveaux de gris différents**.
- En choisissant un **seuil T**, on divise les pixels en deux groupes :
  - ceux dont l'intensité ≤ T (fond → noir),
  - ceux dont l'intensité > T (objet → blanc).

Définition

- Le **seuillage** est une technique de **segmentation** qui consiste à transformer une image en niveaux de gris en une image **binaire** (noir/blanc) ou multi-classes, en fonction d'un ou plusieurs **seuils**, dans l'objectif de séparer **l'avant-plan** (objet d'intérêt) de **l'arrière-plan**.
- Soit une image en niveaux de gris f(x,y).
- Le seuillage binaire s'écrit :

$$g(x,y) = \begin{cases} 255 & \text{si } f(x,y) > T \\ 0 & \text{si } f(x,y) \leq T \end{cases}$$

  - f(x,y) = intensité du pixel en (x,y).
  - T = seuil choisi.
  - g(x,y) = image seuillée (binaire).

Utilité du seuillage

- Extraire des **objets d'intérêt** (lettres, pièces, cellules…).
- Réduire la complexité d'une image (de 256 niveaux → 2 niveaux).
- Préparer une image pour :
  - la **détection de contours**,
  - la **reconnaissance de formes**,
  - l'**analyse de régions**.
- Exemple : Prenons une radiographie :
  - Fond sombre (valeurs basses → pic gauche dans l'histogramme).
  - Os clairs (valeurs hautes → pic droit).
  - Le seuil est choisi **entre les deux pics** → segmentation des os.

Types de seuillage

1. Seuillage global (fixe)
2. Seuillage adaptatif (local)
3. Seuillage multi-niveaux
4. Seuillage automatique (méthodes statistiques)

Seuillage global (fixe)

- **Définition :** Le seuillage global (ou fixe) consiste à appliquer **un seul seuil T** à toute l'image, sans tenir compte de la position des pixels ou des variations d'éclairage.
- Formule :

$$g(x,y) = \begin{cases} 255 & \text{si } f(x,y) > T \\ 0 & \text{sinon} \end{cases}$$

  - f(x,y) = intensité du pixel en (x,y).
  - T = seuil choisi manuellement ou calculé.
  - g(x,y) = image binaire (noir et blanc).

Seuillage global (fixe)

- **Exemple intuitif**
  - Imaginons une pièce sombre sur un fond clair :
  - Si on fixe T = 120 (sur une échelle [0-255]) :
  - Les pixels > 120 (fond clair) deviennent blancs.
  - Les pixels ≤ 120 (pièce sombre) deviennent noirs.

➔ On a séparé la pièce (objet) du fond.

Avantages

- Simple et rapide à mettre en œuvre.
- Suffisant pour des images **bien contrastées**.
- Peu coûteux en calcul.

Limites

- Dépend fortement du choix du seuil T :
  - Si T trop bas → beaucoup de bruit.
  - Si T trop haut → perte d'objet.
- Sensible aux variations d'éclairage :
  - une ombre peut être confondue avec l'objet.
- Inefficace pour des images complexes (multimodales).

Seuillage global (fixe)

Python avec OpenCV

```python
import cv2
import matplotlib.pyplot as plt

# Lire l'image en niveaux de gris
img = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)

# Seuillage global avec T=120
_, binary = cv2.threshold(img, 120, 255, cv2.THRESH_BINARY)
```

Pseudo-code

```
Entrée : Image en niveaux de gris f(x,y), seuil T
Sortie : Image binaire g(x,y)

Pour chaque pixel (x,y) de l'image :
    Si f(x,y) > T :
        g(x,y) = 255
    Sinon :
        g(x,y) = 0
```

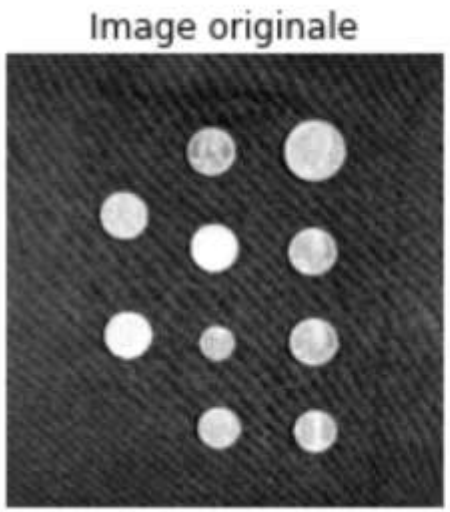
Seuillage global (fixe)

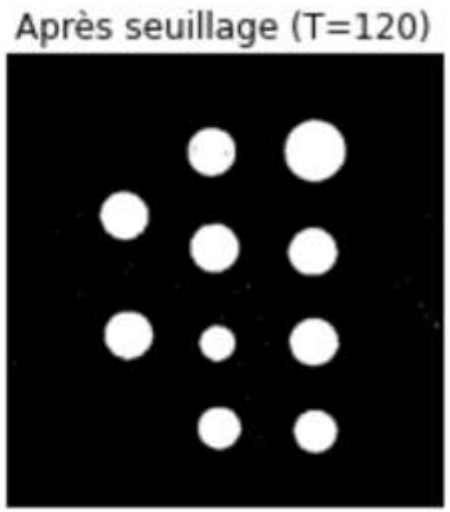
- Exemple

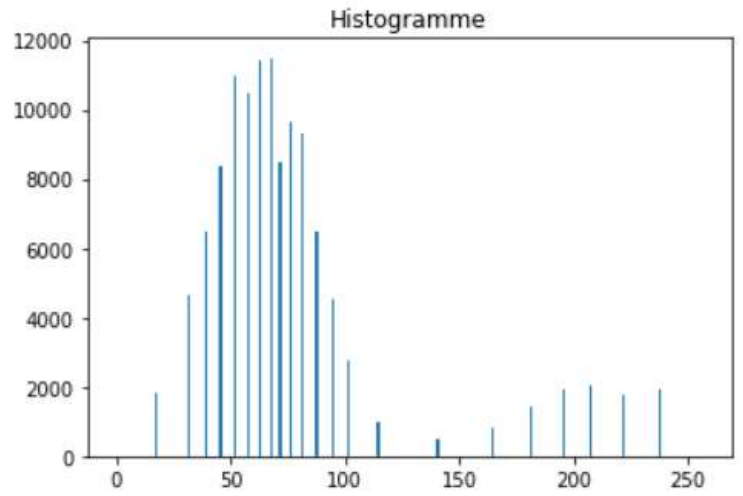
```python
import cv2
import matplotlib.pyplot as plt

# Charger une image en niveaux de gris
img = cv2.imread(r"D:\Cours\img\Seuillage\coins.png", cv2.IMREAD_GRAYSCALE)

# Seuillage global
_, thresh = cv2.threshold(img, 120, 255, cv2.THRESH_BINARY)
```







Seuillage global (fixe)

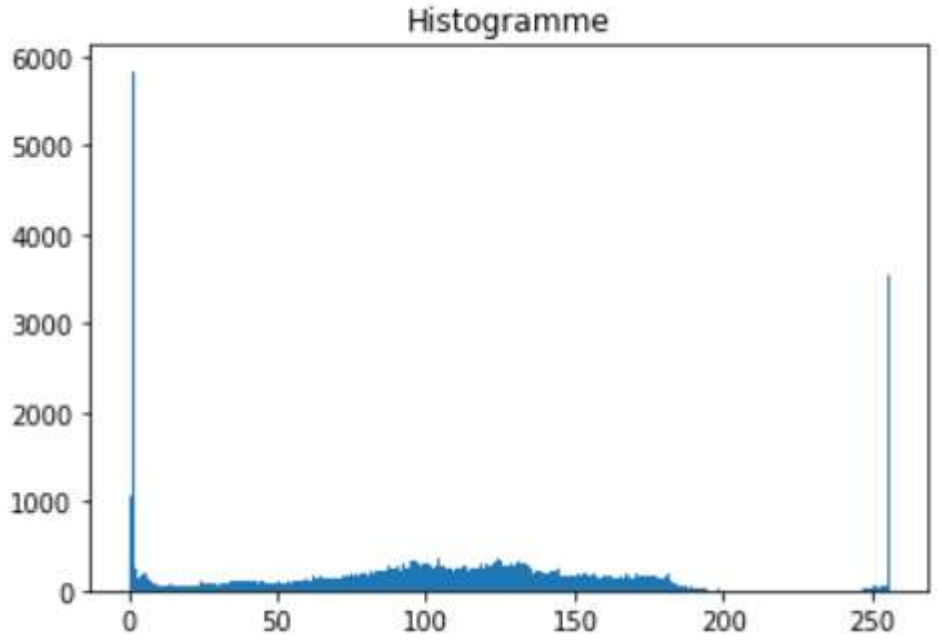
- Limite : Si l'éclairage varie ou si l'objet et le fond ont des intensités proches alors le seuillage global échoue.

Image originale



Après seuillage (T=120)





Seuillage adaptatif (local)

- **Définition :** Le **seuillage adaptatif** consiste à calculer un **seuil différent pour chaque région** de l'image au lieu d'un seuil global unique.
  L'idée : si l'éclairage ou le contraste varient dans l'image, un seul seuil ne suffit pas. Le seuillage adaptatif s'adapte localement.
- **Principe :**
  - L'image est découpée en **petites fenêtres** (blocs locaux).
  - Pour chaque pixel, on calcule un **seuil local** basé sur l'intensité des voisins.
  - On compare la valeur du pixel à ce seuil → résultat binaire (0 ou 255).
- Deux méthodes classiques (implémentées dans OpenCV) :
  - **Moyenne locale** :

    $$T(x,y) = \text{moyenne des intensités des voisins dans une fenêtre centrée en } (x,y) - C$$
  - **Gaussienne pondérée** :

    $$T(x,y) = \text{moyenne pondérée gaussienne des voisins} - C$$

Seuillage adaptatif (local)

```python
# Seuillage adaptatif : méthode Moyenne
th_mean = cv2.adaptiveThreshold(img, 255,
                                cv2.ADAPTIVE_THRESH_MEAN_C,
                                cv2.THRESH_BINARY,
                                15, 10)
```

```python
# Seuillage adaptatif : méthode Gaussienne
th_gauss = cv2.adaptiveThreshold(img, 255,
                                 cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                                 cv2.THRESH_BINARY,
                                 15, 10)
```

- **Exemple d'utilisation :** Imagine un document scanné mal éclairé :
  - En haut de la feuille → éclairage fort, le texte est bien noir.
  - En bas de la feuille → zone sombre, le texte paraît gris.
  - Un seuillage global échoue (le seuil marche en haut mais pas en bas).
  - Un seuillage adaptatif réussit : chaque région trouve son seuil optimal → tout le texte est binarisé correctement.

Seuillage adaptatif (local)

- Exemple

```
# Seuillage adaptatif - Moyenne
th_mean = cv2.adaptiveThreshold(img, 255,
                                cv2.ADAPTIVE_THRESH_MEAN_C,
                                cv2.THRESH_BINARY,
                                15, 10)

# Seuillage adaptatif - Gaussien
th_gauss = cv2.adaptiveThreshold(img, 255,
                                 cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                                 cv2.THRESH_BINARY,
                                 15, 10)
```

Image originale



Adaptatif Moyenne (15,10)



Adaptatif Gaussien (15,10)



Seuillage adaptatif (local)

- Limite : si l'image contient plusieurs objets avec des intensités très différentes, un seul type de calcul local (moyenne ou Gaussienne) peut ne pas séparer correctement tous les objets.



Seuillage multi-niveaux

- **Définition :** Le **seuillage multi-niveaux** est une extension du seuillage global classique.
- Au lieu d'un **seuil unique T**, on choisit **plusieurs seuils** :

$$T1, T2, \ldots, Tn$$

- L'image est ainsi découpée en **plusieurs classes d'intensités**.
- Exemple avec deux seuils T1 et T2 :
  - Si $I(x,y) < T1 \rightarrow$ pixel noir (classe 1).
  - Si $T1 \leq I(x,y) < T2 \rightarrow$ pixel gris (classe 2).
  - Si $I(x,y) \geq T2 \rightarrow$ pixel blanc (classe 3).

Seuillage multi-niveaux

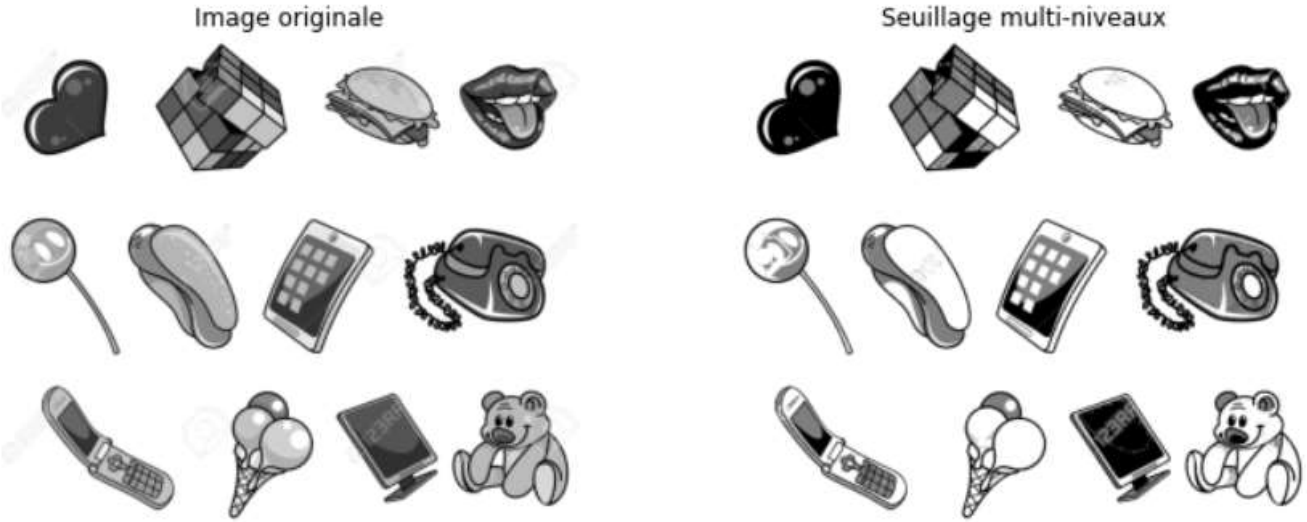
- **Principe**
  - Le seuillage global sépare seulement **objet vs fond**.
  - Le seuillage multi-niveaux permet de séparer **plusieurs régions d'intérêt**.
- **Exemple** :
  - Radiographie → séparer **air (noir)**, **tissus mous (gris)**, **os (blanc)**.
  - Photo satellite → distinguer **eau, végétation, sol**.
- **Avantages**
  - Permet de distinguer plusieurs objets / zones dans une seule image.
  - Très utile pour des **images médicales**, **satellites**, ou **industrielles**.
- **Limites**
  - Choix des seuils T1,T2,… pas toujours évident.
  - Si les classes d'intensité ne sont pas bien séparées, ça donne des erreurs.
  - Sensible au bruit → nécessite parfois un **lissage préalable** (filtre Gaussien).

Seuillage multi-niveaux

- Exemple

```
# 2. Définir les seuils manuellement
seuil1 = 85
seuil2 = 170

# 3. Appliquer le seuillage multi-niveaux
thresholded = np.zeros_like(img)
thresholded[img < seuil1] = 0           # niveau 0
thresholded[(img >= seuil1) & (img < seuil2)] = 127  # niveau intermédiaire
thresholded[img >= seuil2] = 255      # niveau maximal
```



Seuillage automatique

- **Définition :** Le **seuillage automatique** est une technique qui **détermine le seuil tout seul**, sans que l'utilisateur ait besoin de le fixer manuellement.
- C'est utile car on n'a pas toujours une idée du bon seuil (surtout si l'image change de luminosité).

Seuillage automatique

cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

- **Méthodes courantes**
  1. **Méthode d'Otsu (la plus célèbre)**
     - Elle choisit le seuil T qui minimise la variance intra-classes (pixels de même classe sont le plus homogènes possible).
     - Autrement dit, Otsu cherche le seuil qui sépare au mieux deux classes : objet et fond.
     - Étapes simplifiées :
       - On calcule l'histogramme de l'image.
       - On teste tous les seuils possibles.
       - On garde celui qui minimise la variance interne (ou maximise la variance entre classes).
  2. **Autres méthodes**
     - **Entropie de Kapur** : choisit le seuil qui maximise l'entropie des classes.
     - **IsoData / Méthode itérative** : ajuste le seuil à partir de la moyenne des intensités des deux classes jusqu'à stabilisation.

Seuillage automatique

- Avantages
  - Pas besoin de choisir
  - T soi-même.
  - Rapide et robuste si l'histogramme est bimodal (2 pics bien séparés).
- Limites
  - Suppose que l'image est composée de 2 classes principales → si l'image contient plusieurs objets → résultats moyens.
  - Sensible au bruit et aux variations d'éclairage → nécessite parfois un filtrage avant (ex. flou gaussien).
- En résumé :
  - Seuillage manuel → on fixe T à la main.
  - Seuillage automatique (Otsu, etc.) → l'algorithme calcule T de façon optimale.

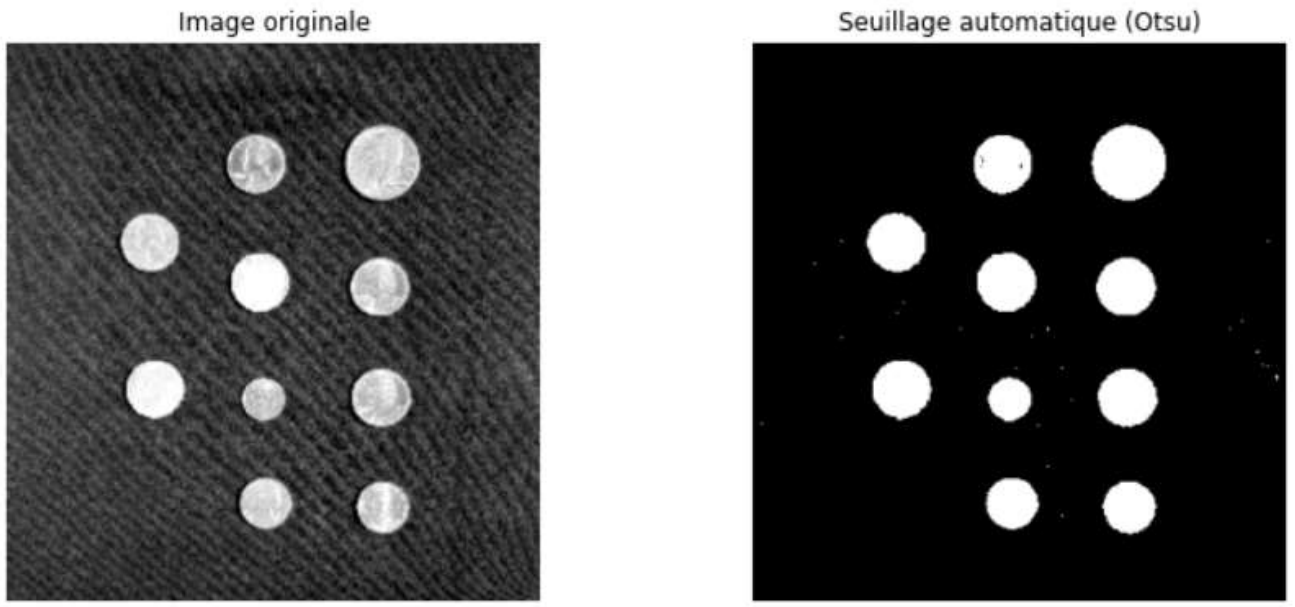
Seuillage automatique

- Exemple

```python
# 2. Appliquer le seuillage automatique d'Otsu
# cv2.THRESH_BINARY: pixels > seuil → 255, sinon 0
ret, otsu_thresh = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

print("Seuil calculé automatiquement par Otsu :", ret)
```

```
Seuil calculé automatiquement par Otsu : 114.0
```



Seuillage et projets proposés

1. Détection précoce des maladies des plantes

- **Problème** : distinguer une feuille saine d'une feuille malade (taches, décolorations).
- **Utilité du seuillage** :
  - Segmenter les zones suspectes (plus sombres ou plus claires que le reste de la feuille).
  - Binariser l'image pour isoler les régions anormales avant une analyse de texture ou de couleur.
- **Exemple** : seuillage sur le canal vert pour séparer feuille saine (vert uniforme) et zones malades (couleurs différentes).

Seuillage et projets proposés

2. Analyse d'images médicales pour la prévention de maladies

- **Problème** : détecter une anomalie (tumeur, calcification, masse anormale).
- **Utilité du seuillage** :
  - Isoler les zones d'intérêt en fonction de leur intensité (par ex. os clairs vs tissus mous sombres dans une radiographie).
  - Préparer une segmentation plus avancée (watershed, contours, IA).
- **Exemple** : seuillage dans une IRM pour séparer le tissu cérébral normal d'une zone suspecte.

Seuillage et projets proposés

3. Surveillance de la pollution et des déchets plastiques

- **Problème** : détecter des plastiques flottants dans l'eau ou sur une plage.
- **Utilité du seuillage** :
  - Mettre en évidence des objets brillants ou de couleur différente du fond (plastique blanc sur eau sombre).
  - Binariser pour compter ou mesurer la surface des déchets.
- **Exemple** : seuillage couleur (HSV) pour isoler les plastiques transparents ou blancs.

Seuillage et projets proposés

4. Accessibilité pour personnes malvoyantes

- **Problème** : aider à reconnaître des panneaux, du texte ou des obstacles.
- **Utilité du seuillage** :
  - Transformer une image en noir/blanc très contrastée pour lecture simplifiée.
  - Extraire le texte (OCR) après seuillage.
- **Exemple** : prendre une image de texte, appliquer un seuillage pour obtenir un contraste maximal → lecture vocale facilitée.

Seuillage et projets proposés

5. Surveillance intelligente des foules et modélisation des comportements collectifs

- **Problème** : détecter et suivre des personnes dans une foule.
- **Utilité du seuillage** :
  - Première étape de détection : seuillage après soustraction de fond pour isoler les silhouettes.
  - Ensuite, détection de contours ou suivi d'objets.
- **Exemple** : une caméra fixe → seuillage des zones en mouvement pour obtenir une carte binaire des personnes.

A retenir

Le seuillage n’est **jamais la solution finale**, mais c’est **la porte d’entrée** dans la segmentation. Il simplifie l’image pour permettre des traitements plus avancés (morphologie, IA, deep learning).